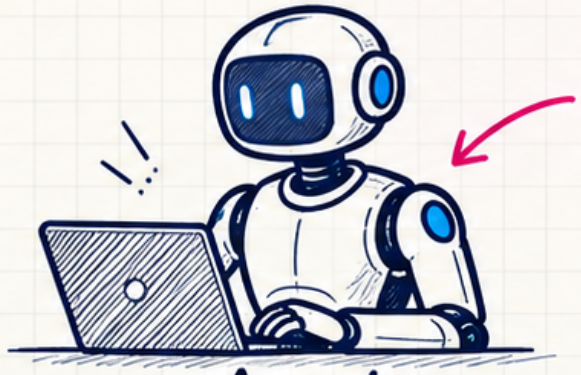
Agents are new users of the platform

The platform is reused. The agent becomes a principal.



Developer

- Identity
- Permissions
- Audit trail
- Paved roads



Agent

- Identity
- Permissions
- Audit trail
- Paved roads

Human approval for risky actions

Governance decides what agents may do.

Existing developer platform

In the reference: own credential, scoped tools, recorded actions.

AGENTIC DEVELOPER PLATFORM

Three layers

Build bottom-up. Reuse what already works.

3 Agent infrastructure

What makes agents safe to run

- Runtime (alpha)
- Identity
- Capability
- Approval
- Tools
- Memory
- Audit
- Evaluation

2 Paved roads

How work flows

- Scaffolder
- Dependency updates
- PR triage
- Scheduled work

1 Developer platform

What you need anyway

- GitOps
- Infrastructure as code
- Secrets
- Edge
- Policy
- Data & backups
- Observability
- Developer portal

Layer 3 reuses the secrets, deployment and docs systems below.

In the reference: Kubernetes • GitOps • scoped credentials

A change stops at the gate

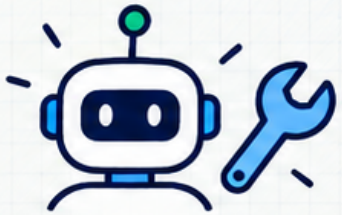
Delete the ConfigMap approval-gate-probe.

A human sees the exact call and arguments.

1 **Gateway**
Same entry point



2 **Operator agent**
Mutating tools



3 **Checks first**
Read object + throwaway label



4 **Human approval**



Approve

5 **Action + record**
Only after approval



CI validator

Admission policy

Runtime binding





Reject: no action, no automatic retry.

Three enforcement layers. No prompt can grant approval.

In the reference: `requireApproval: true`

The approval gate, both ways

Same request, run twice: rejected, then approved.

Delete approval-gate-probe

Read object and confirm throwaway label

Waiting for a human

k8s_delete_resource requires approval

Run 1 • Reject



Object remains

"The delete didn't go through. The approval step rejected the call, so the ConfigMap is still in the default namespace. I haven't retried."

Run 2 • Approve



Object deleted

Call + decision recorded

"The ConfigMap approval-gate-probe is deleted from the default namespace. A follow-up get on it failed, confirming it is gone."

A protocol-level pause. Rejection ends that call.

Six controls around the agent

Enforce permissions outside the model.

01 Identity

Scoped credentials.
Keys injected at egress.



02 Capability

Read, write, destructive.
Mutating bindings require approval.



03 Audit

Record calls and decisions.
Alert when reporting stops.





04 Evaluation

Known-good answers.
Rules, then classifier, then judge.



05 Isolation

Sandbox per conversation.
Default-deny egress.



06 Change control

Tools, models and permissions change through reviewed Git.



The model cannot grant itself permissions.

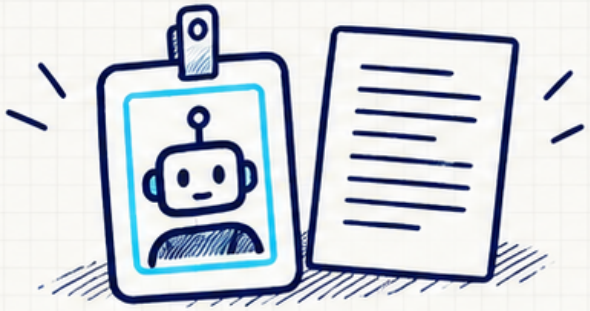
Reference runtime: alpha

What maps to a company

Reuse the governance. Decide the ownership.

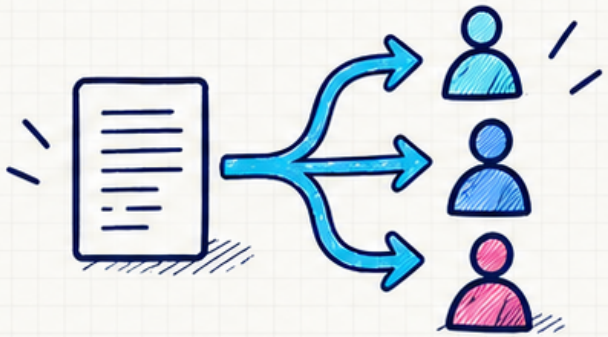
01 Maps directly

- Agents as principals
- Team-owned tool taxonomy
- Audit feed to your SIEM
- Shared docs contract



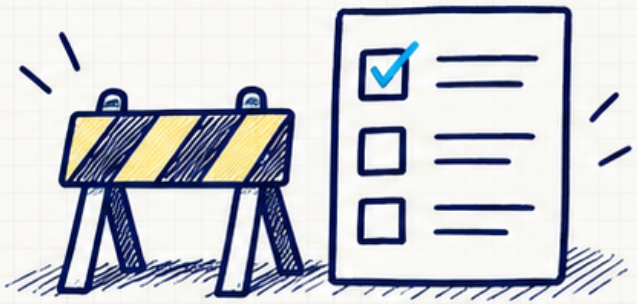
02 Needs a decision

- Who approves each write?
- Route by team and risk
- Name the accountable owner



03 Still early

- Sandboxed runtime is alpha
- Domain evaluation needs work



Start here

One team → One read-only agent → Every call recorded

Answers from the team's docs and systems. Cannot change them.

Start read-only. Earn write access with evidence.

Five questions for the group

Decide the operating model before expanding access.

1 Reuse

Which platform pieces already exist, and which are missing?



2 Authority

Who classifies tools, and who approves a write?



3 First agent

What is the first read-only agent, and whose docs would it read?



4 Accountability

What does the audit record feed, and who reads it?



5 Evaluation

How do we evaluate before and after a model change?



The agents are visible. The platform makes them safe to run.

The reference can demonstrate a question, an approval and a refusal.